Sentiment Classification with TF-IDF

Structured Notes

Word2Vec Limitations, TF-IDF

Vectorization,
Naive Bayes & SVM Classifiers, Model Evaluation

Contents

1 From Word Embeddings to Document Features **2**
1.1 Word2Vec Recap . . . 2
1.2 Why Word2Vec Alone Is Not Enough for Classification . . . 2
1.3 The Naive Averaging Problem . . . 2
1.4 Word2Vec's Role in the Bigger Picture . . . 2

2 TF-IDF: Term Frequency – Inverse Document Frequency **3**
2.1 What Is TF-IDF? . . . 3
2.2 How TF-IDF Assigns Importance . . . 3
2.3 TF-IDF vs Word2Vec Comparison . . . 4

3 Pre-processing for Sentiment Classification **4**
3.1 Task-Dependent Pre-processing . . . 4
3.2 The Stop Word Problem in Sentiment Analysis . . . 4
3.3 Modified Pre-processing Pipeline . . . 4

4 Building the Sentiment Dataset **5**
4.1 Dataset Structure . . . 5
4.2 Sample Reviews . . . 5
4.3 Applying Pre-processing . . . 6

5 TF-IDF Vectorization **6**
5.1 Using Scikit-learn's TfidfVectorizer . . . 6
5.2 Inspecting the Vocabulary . . . 7
5.3 Top Weighted Words Example . . . 7
5.4 Observations on TF-IDF . . . 7

6 Training Classical ML Classifiers **7**
6.1 Model Selection . . . 7
6.2 Train-Test Split . . . 8
6.3 Training the Models . . . 8
6.4 Sample Predictions . . . 8

7 Model Evaluation **9**
7.1 Why Evaluate on Unseen Data . . . 9
7.2 Confusion Matrix . . . 9
7.3 Understanding the Results . . . 9
7.4 Why SVM Is Strong for Text Classification . . . 10
7.5 Why Naive Bayes Is Still Useful . . . 10

8 Summary **10**

1 From Word Embeddings to Document Features

1.1 Word2Vec Recap

Word2Vec provides dense vector representations for individual words. The key properties of Word2Vec embeddings are:

- Words can be converted into **dense vectors**
- Semantically similar words appear close together in vector space
- The vector space captures contextual relationships between words

These embeddings are powerful for understanding word-level semantics, and they form the foundation of modern NLP systems including RNNs, LSTMs, GRUs, and Transformers.

1.2 Why Word2Vec Alone Is Not Enough for Classification

For classification tasks, the entire sentence or review must be represented as a single input to the machine learning model — not just individual words.

The Core Problem

Word2Vec gives vectors for individual **words** (e.g., "product", "good", "not"). However, classifiers need features for **whole documents** or sentences. This creates a fundamental mismatch when using classical ML models for text classification.

Word2Vec embeddings are designed for word-level representation. For classification, we need sentence-level or document-level features. This raises the question: how do we bridge the gap from word vectors to document vectors?

1.3 The Naive Averaging Problem

A common beginner approach is to take the average of all word vectors in a sentence to create a single sentence-level vector. While simple, this approach has significant problems:

- **Word order is lost**: The average treats all words equally regardless of position
- **Negation becomes weak**: Words like "not" get diluted when averaged with other words
- **All words are treated almost equally**: Important sentiment words lose their significance

Negation Dilution Example

Consider two sentences:

- "This product is good"
- "This product is **not** good"

After naive averaging (giving equal weight to each word), these two sentences may become **too similar** in vector space, even though their sentiments are opposite. The word "not" gets diluted when averaged with all other word vectors.

This is one reason why **sequence-aware models** like RNN, LSTM, GRU, and Transformers became important — they can process word order and capture contextual dependencies that naive averaging destroys.

1.4 Word2Vec's Role in the Bigger Picture

Word2Vec is very useful for:

- Semantic understanding and similarity tasks
- Building intuition about word embeddings
- Deep learning models (RNN, LSTM, Transformers) that can process sequences

However, for **classical sentiment classifiers** (models that existed before deep learning), Word2Vec is not the easiest baseline because:

- It works at the word level only
- Sentence representation is not straightforward
- Naive averaging can lose important meaning and word order

Key Distinction

Word2Vec introduces the concept of embeddings, which are the central point in modern NLP (Transformers, LLMs). But for a first text classifier using classical ML, **TF-IDF** is the more practical and appropriate choice.

2 TF-IDF: Term Frequency – Inverse Document Frequency

2.1 What Is TF-IDF?

TF-IDF helps represent a whole sentence or document as a numerical feature vector. It addresses two questions:

- Which words appear in the review?
- Which words are **important** in that review versus being common across all reviews?

TF (Term Frequency): How often a word appears in a document.

$$\text{TF}(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

IDF (Inverse Document Frequency): How rare or informative that word is across the whole dataset.

$$\text{IDF}(t, D) = \log\left(\frac{\text{Total number of documents in } D}{\text{Number of documents containing term } t}\right)$$

TF-IDF Score:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

2.2 How TF-IDF Assigns Importance

A word gets **higher importance** if it appears in a specific document but not in every document. Words that appear in almost all documents (like “the”, “is”, “and”) receive lower TF-IDF scores because they are less informative for distinguishing one document from another.

Sentiment Signal Words

Words like **excellent**, **worst**, **rude**, **defective**, **smooth**, and **delayed** can become strong sentiment signals. These words tend to appear in specific types of reviews (positive or negative) rather than uniformly across all reviews, giving them high TF-IDF scores.

Sarcasm Limitation

TF-IDF relies on the presence of positive or negative words to classify sentiment. If sarcasm is used (e.g., positive words in a negative context), basic TF-IDF classifiers will fail. Handling sarcasm requires more advanced deep learning models that can understand contextual nuance.

2.3 TF-IDF vs Word2Vec Comparison

Aspect	Word2Vec	TF-IDF
Vector type	Dense vectors	Sparse vectors
Representation level	Word-level	Document/sentence-level
Semantic meaning	Captures semantic relationships	Captures word importance
Best suited for	Sequential models (RNN, LSTM, Transformers)	Classical ML (Naive Bayes, SVM)
Interpretability	Less interpretable (neural)	More interpretable
New word generation	Possible with sequential models	Cannot generate new words

Key Takeaway

Word2Vec gave us **dense vectors for words**. TF-IDF gives us **sparse vectors for whole reviews**. For classical text classification, TF-IDF provides the document-level features that classifiers require.

3 Pre-processing for Sentiment Classification

3.1 Task-Dependent Pre-processing

Pre-processing should always be **task-dependent**. Different tasks require different information to be preserved. For sentiment classification specifically, standard pre-processing steps need modification.

Sentiment Pre-processing Principle

In sentiment classification, removing negation words like “not”, “no”, “never” can be harmful because they change the meaning of a sentence entirely. Standard stop-word removal treats these as stop words and removes them, which destroys critical sentiment information.

3.2 The Stop Word Problem in Sentiment Analysis

Consider the standard NLP stop word list, which includes words like “not”, “no”, “nor”:

- “The product is good” → After stop word removal: “product good” (Positive)
- “The product is **not** good” → After stop word removal: “product good” (Still appears Positive!)

Both sentences produce nearly identical processed text after standard stop word removal, even though their sentiments are opposite. This demonstrates why **pre-processing must be adapted** for sentiment tasks.

3.3 Modified Pre-processing Pipeline

The adapted pre-processing pipeline for sentiment classification includes:

1. Convert text to lowercase
2. Remove extra spaces and special characters
3. Remove numbers (rarely informative in review text)
4. Remove stop words **but keep negation words**
5. Apply POS-aware lemmatization

Sentiment Pre-processing Function

```python
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
import re

def preprocess_for_sentiment(text):
    # Lowercase
    text = text.lower()
    # Remove numbers
    text = re.sub(r'\d+', '', text)
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    # Define negation words to keep
    negation_words = {'not', 'no', 'nor', 'never',
                      'neither', 'none'}

    # Remove stop words but preserve negation
    stop_words = set(stopwords.words('english'))
    stop_words -= negation_words

    tokens = text.split()
    tokens = [w for w in tokens if w not in stop_words]

    # Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(w) for w in tokens]

    return ' '.join(tokens)
```

Listing 1: Pre-processing pipeline adapted for sentiment classification

POS-Aware Lemmatization

POS (Part of Speech) aware lemmatization ensures that words are reduced to their correct base form based on their grammatical role. This produces better results than simple lemmatization because it considers whether a word is a noun, verb, adjective, etc., before reducing it.

4 Building the Sentiment Dataset

4.1 Dataset Structure

For training a sentiment classifier, two components are needed:

- **Input text**: The review or sentence
- **Target label**: Positive or negative (binary classification)

The dataset used contains 80 rows with balanced class distribution (40 positive, 40 negative) and 2 columns (text and sentiment label).

4.2 Sample Reviews

Positive reviews:

- “The delivery was quick and packaging was excellent”
- “Customer support resolved my issue very quickly”
- “I really love the fabric quality and fitting”
- “Refund was processed smoothly on time”
- “Amazing product totally worth the price”

Negative reviews:

- "My order arrived two days late and the box was damaged"
- "Payment failed twice and money got deducted from my account"
- "The product quality is terrible and feels very cheap"
- "The app crashed during checkout and I could not place the order"

4.3 Applying Pre-processing

Applying to DataFrame

```python
import pandas as pd

# Apply pre-processing to all reviews
df['processed_tokens'] = df['text'].apply(
    preprocess_for_sentiment
)

# Example output:
# Text: "My order arrived two days late and the box
#        was damaged"
# Processed: "order arrive two day late box damage"
# Sentiment: negative
```

Listing 2: Applying sentiment pre-processing to the dataset

After pre-processing, the important observation is that negation words are preserved while other stop words are removed. The processed text retains the essential sentiment information needed for classification.

5 TF-IDF Vectorization

5.1 Using Scikit-learn's TfidfVectorizer

The `TfidfVectorizer` from scikit-learn converts the processed text into TF-IDF feature vectors:

TF-IDF Vectorization

```python
from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF vectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit and transform the training text
X_tfidf = tfidf_vectorizer.fit_transform(X_text)

print(f"TF-IDF matrix shape: {X_tfidf.shape}")
# Output: (80, 234)
# 80 documents, 234 unique features
```

Listing 3: Creating TF-IDF features using scikit-learn

fit_transform vs transform

- `fit_transform()`: Used **only during training**. It learns the vocabulary and computes IDF values, then transforms the data.
- `transform()`: Used for **both training and testing**. It applies the already-learned vocabulary and IDF values to convert new text into vectors.

> The testing data must be transformed using the same vocabulary learned from training data to ensure consistent feature dimensions.

5.2 Inspecting the Vocabulary

Feature Inspection

```
feature_names = tfidf_vectorizer.get_feature_names_out()
print(f"Number of TF-IDF features: {len(feature_names)}")
# Output: 234

print(f"Sample features: {feature_names[:6]}")
# Output: ['account', 'agent', 'already',
#          'amaze', 'app', 'arrive']
```

Listing 4: Inspecting TF-IDF features

The TF-IDF matrix is analogous to one-hot encoding, but at the sentence level rather than the word level. Each sentence gets a vector of length 234 (the total vocabulary size), where most values are zero (sparse) and non-zero values indicate the TF-IDF weight of that word in that sentence.

5.3 Top Weighted Words Example

For the review "My order arrived two days late and the box was damaged":

- Processed text: "order arrive two day late box damage"
- Top TF-IDF features (highest weights): `box`, `damage`, `late`, `day`, `arrive`, `order`

Every sentence receives features from all 234 vocabulary words, but only the words present in that sentence have non-zero TF-IDF scores.

5.4 Observations on TF-IDF

- TF-IDF converts each review into a **numerical feature vector**
- Important words receive **higher weights**
- The classifier can learn patterns: positive reviews often contain words like "excellent", "smooth", "resolved"; negative reviews contain words like "damaged", "failed", "terrible", "frustrating"
- TF-IDF turns text into structured input for classical ML models

Curse of Dimensionality

With larger datasets, the number of unique words (and therefore TF-IDF features) can grow very large — potentially to thousands or even millions. This creates sparse, high-dimensional vectors, which is known as the **curse of dimensionality**. For the demonstration dataset, 234 features is manageable, but real-world datasets may require dimensionality reduction techniques.

6 Training Classical ML Classifiers

6.1 Model Selection

With TF-IDF features prepared, classical machine learning models can be trained. Two models are used:

1. **Multinomial Naive Bayes**: Simple and fast, based on prior and posterior probabilities (Bayes' theorem from the 18th century)
2. **Support Vector Machine (Linear SVM)**: Often very strong for text classification, works well on sparse TF-IDF features by maximizing the margin between classes

Additional models that can be used include logistic regression and decision trees. For binary classification

tasks like sentiment analysis, these simpler models are often sufficient.

Kernel SVM Option

Instead of Linear SVM, a kernel SVM (e.g., RBF kernel) can also be used, which adds some non-linearity to the decision boundary. However, for sparse TF-IDF features, linear SVM typically performs well.

6.2 Train-Test Split

Data Splitting

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_tfidf, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # Maintains class balance
)
# 80% training (64 samples), 20% testing (16 samples)
```

Listing 5: Splitting data with stratification

Stratified sampling ensures that the class distribution in the training and testing sets matches the original dataset distribution. With balanced data (40 positive, 40 negative), stratification ensures both splits maintain this 50/50 ratio.

6.3 Training the Models

Model Training

```python
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC

# Train Naive Bayes
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)

# Train SVM
svm_model = LinearSVC()
svm_model.fit(X_train, y_train)
```

Listing 6: Training Naive Bayes and SVM classifiers

Classical ML models train extremely fast compared to deep learning models. With 80 data points and 234 features, both models train almost instantaneously.

6.4 Sample Predictions

Review	Naive Bayes	SVM
"The product quality is amazing and I'm very happy"	Positive	Positive
"Delivery was late"	Negative	Negative

Both models can correctly predict sentiment from TF-IDF features. With more complex or ambiguous text, differences between models may emerge.

7 Model Evaluation

7.1 Why Evaluate on Unseen Data

After training a classifier, it is essential to measure how well it performs on **unseen data** (the test set). This tests whether the model generalizes beyond the training examples. The evaluation uses:

- **Accuracy**: Overall percentage of correct predictions
- **Confusion Matrix**: Breakdown of true positives, true negatives, false positives, false negatives
- **Classification Report**: Precision, recall, and F1-score for each class

7.2 Confusion Matrix

Evaluation Code

```
from sklearn.metrics import (confusion_matrix,
                             classification_report,
                             accuracy_score)

# Naive Bayes evaluation
y_pred_nb = nb_model.predict(X_test)
print("Naive Bayes Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_nb,
                       labels=['positive', 'negative']))

# SVM evaluation
y_pred_svm = svm_model.predict(X_test)
print("SVM Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_svm,
                       labels=['positive', 'negative']))

# Classification reports
print(classification_report(y_test, y_pred_nb))
print(classification_report(y_test, y_pred_svm))
```

Listing 7: Model evaluation with confusion matrix and classification report

7.3 Understanding the Results

With 16 test samples (8 positive, 8 negative on a balanced dataset), both models achieved high accuracy on this small synthetic dataset.

Evaluation Metrics

- **Precision**: Of all samples predicted as a class, how many actually belong to that class?

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall**: Of all actual samples of a class, how many were correctly predicted?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- **F1-Score**: Harmonic mean of precision and recall

$$\text{F1} = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

7.4 Why SVM Is Strong for Text Classification

SVM handles high-dimensional sparse data features well. Since TF-IDF produces sparse vectors in a high-dimensional space, SVM is particularly well-suited because:

- It finds the **maximum margin** hyperplane that separates classes
- It works effectively with sparse feature representations
- With just two classes (binary classification), SVM is highly efficient

7.5 Why Naive Bayes Is Still Useful

Despite being a simple algorithm, Naive Bayes remains useful because:

- It is **fast** and computationally inexpensive
- It performs **equivalently to SVM** on many text classification tasks
- It is based on Bayes' theorem (prior and posterior probabilities), a fundamental concept from the 18th century that remains foundational to machine learning
- It serves as a strong **baseline** against which more complex models are compared

8 Summary

- **Word2Vec** provides dense word-level embeddings that capture semantic similarity, but it is not directly suitable for classical ML classifiers that need document-level features
- **Naive averaging** of word vectors loses word order, dilutes negation, and treats all words equally — motivating the need for sequence-aware models (RNN, LSTM) or document-level features (TF-IDF)
- **TF-IDF** converts entire sentences/documents into sparse numerical feature vectors by weighting words based on their frequency in a document versus their rarity across all documents
- **Pre-processing for sentiment** must be task-dependent: negation words (not, no, nor, never, neither, none) must be preserved because they change sentence meaning
- **TF-IDF Vectorization** using scikit-learn's `TfidfVectorizer` creates a sparse matrix where each document is represented by the TF-IDF weights of all vocabulary words
- **Multinomial Naive Bayes** is simple and fast, while **Linear SVM** is particularly strong for sparse TF-IDF features in text classification
- **Stratified train-test splitting** maintains class balance between training and testing sets
- **Model evaluation** using confusion matrices and classification reports (precision, recall, F1-score) measures generalization on unseen data
- TF-IDF is **more interpretable** than Word2Vec and surprisingly strong for sentiment baselines, though it fails for long paragraphs and sarcasm where sequential models excel